# How Robust Is Character-Level Transliteration to Informal Typing? A Paired Noise Benchmark and Augmentation Study for Romanized Tamil

Lebi Raja
*Division of Data Science and Cybersecurity*
*Karunya Institute of Technology and Sciences*
India

KRM Lohith
*Division of Data Science and Cybersecurity*
*Karunya Institute of Technology and Sciences*
India

Allen John Isac
*Division of Data Science and Cybersecurity*
*Karunya Institute of Technology and Sciences*
India

KP Shohil
*Division of Data Science and Cybersecurity*
*Karunya Institute of Technology and Sciences*
India

***Abstract*—Tamil speakers routinely type Tamil in Latin letters ("Tanglish") and software must convert it back to Tamil script, yet published transliteration models are trained and tested on carefully romanized text. We measure what happens under informal typing for word-level Latin-to-Tamil transliteration with character-level sequence-to-sequence models. We release a seeded, paired benchmark of seven Tamil-specific noise operators at four severities (28 noisy copies of the Dakshina test lexicon, each item tied to its clean form and references) and train 110 models: a BiLSTM with attention and a pre-LN Transformer at about 1M, 3M and 8M parameters with five seeds each, and eight training conditions with five seeds each for both architectures. Our implementation reproduces the published Dakshina development CER of 8.4. Vowel dropping and keyboard slips are the damaging operators: at $p$=0.3 they raise test CER from about 9 to 22–24, while vowel-length, doubled-consonant and letter-stretching noise cost only 1–2 points. Training with Tamil-phonological noise cuts vowel-drop CER from 24.0 to 17.7 for the BiLSTM, but does not transfer to an operator left out of training (25.0), and costs about 0.8 clean CER. Adding sampled back-transliterations (A4) gives the lowest mixed-noise CER (21.8 against 25.5) at no clean cost, although the clean gain of back-transliteration alone is explained by extra words, not by spelling variety. At matched size the two architectures tie on clean text, and the Transformer loses 0.3–1.3 CER less under heavy mixed noise. Fine-tuning the best model on 2.4M pairs with all six noise types gives one 12.6 MB model that cuts mixed-noise CER from 21.8 to 17.8 at unchanged clean accuracy; real chat typing is only checked preliminarily.**

***Index Terms*—transliteration, Tamil, robustness, data augmentation, sequence-to-sequence, romanization**

## I. Introduction

Most Tamil speakers type on phones using the Latin alphabet. A message such as *vanakkam, eppadi irukkinga?* (வணக்கம், எப்படி இருக்கிறீர்கள்?) must be converted to Tamil script for search, autocomplete or display. This is transliteration: sounds are mapped to letters, not meanings to words. The Latin alphabet cannot express several Tamil distinctions (vowel length, doubled consonants, and the several n, l and r sounds), so writers spell the same word in many ways; Dakshina annotators produced 2.7 romanizations per word on average [1]. Fast typing adds more variation: dropped vowels (*kankl* for *kanakkil*), shortened long vowels, collapsed double letters, swapped digraphs, stretched letters (*semmaaa*) and key slips [2], [3].

Neural transliteration is well studied on clean data. Small character-level BiLSTM and Transformer models reach about 8–9% character error rate (CER) on the Dakshina Tamil lexicon [1], [4], and large mined corpora lift accuracy further [5]. The Tamil text in these corpora, however, is formal (Wikipedia or mined web text) and was romanized word by word with care. Evidence from sister languages and from small noisy test sets suggests that clean-trained systems degrade sharply on informal input: Malayalam CER rises from 7.4 to 22.7 when vowels are dropped [6], and every system, including large language models, falls to about 20% top-1 on 100 hand-noised Tamil words [7]. What is missing is a *controlled* measurement for Tamil: paired clean and noisy versions of the same test words, per-phenomenon breakdowns, severity levels, and a comparison of training remedies and architectures under the same noise.

The architecture question is also open. On Dakshina Tamil an LSTM was better than a Transformer in one study [1] and worse in another [4]; Transformers depend on batch size [8] and often win only with more parameters [9]. No comparison has tested noisy input.

**Contributions.** (1) A paired, seeded robustness benchmark for Tamil: seven noise operators at four severities applied to the 6,833 distinct Dakshina test inputs, with the realised edit rate of every set reported. (2) An augmentation study with six train-

ing conditions and two held-out-operator variants, tested on every noise type, on independent Aksharantar sets and against a more-data control that separates "more words" from "more spelling variety". (3) A parameter-matched, equal-tuning (20 random-search trials) comparison of BiLSTM-attention and Transformer at three sizes with five seeds, paired bootstrap and Kruskal–Wallis tests. (4) A reproducibility package with code, configurations, per-run logs and data checksums, and a deployed single-file model with a typing demo.

**Findings in brief.** Vowel dropping and keyboard slips cause most of the damage (Section VI-B). Tamil-aware noise training removes most of the vowel-drop damage but only for the operators it models (Section VI-D). Extra romanized words help clean accuracy as much as sampled spelling variants do, and the variants add a small robustness gain. The two architectures are close, with the Transformer slightly more robust to heavy mixed noise (Section VI-C). Real chat typing is only checked preliminarily, so the link between synthetic and real noise remains open (Section IX).

## II. Related Work

**Datasets and systems.** Dakshina provides Tamil romanization lexicons and romanized Wikipedia sentences for twelve South Asian languages [1]. Aksharantar and the IndicXlit model extend coverage to 21 languages with 26 million pairs, mostly mined from monolingual and parallel corpora, and report a rise in Tamil top-1 accuracy from 56.0 to 67.7 when mined data are added [5]. Earlier systems include rule and statistical models such as Brahmi-Net [10], multilingual transliteration [11], [12] and the NEWS shared tasks whose metrics we follow [13]. Context-aware models, ensembles and byte-level pretrained models give further gains on sentences [4], [14], and large language models are competitive on clean Tamil [7]. Production keyboards use small finite-state models under tight latency budgets [15].

**Informal and noisy romanization.** Wolf-Sonkin et al. describe variation in Latin-script input across South Asian languages, including optional aspiration and vowel doubling [2]. For Tamil, a shorthand-typing study derives rules from a survey of 215 users but evaluates no neural model [3]; a shared task on ad-hoc romanization shows that all systems degrade [16]; and for Malayalam a BiLSTM loses about 15 CER points on vowel-omitted input [6]. Code-mixed Tamil corpora such as TamilMixSentiment contain real informal text but have no Tamil-script references [17]. Lexicon snapping and language-model reranking can recover accuracy on noisy input [18], [19].

**Noise robustness and augmentation.** Character models are brittle to noise they were not trained on, and robustness to one noise type transfers poorly to others [20]. A balanced mixture of deletions, insertions, substitutions and swaps improves robustness to typos at small clean cost but does not help stylistic variation [21]. For romanized language identification, synthetic data that reproduces human spelling variation beats larger models, whereas machine-romanized data overstate accuracy [22], [23], [24]. Back-translation is the standard way of creating synthetic parallel data [25], and we use its transliteration analogue. Byte-level models are intrinsically more robust to character noise than subword models [14], [26].

**Architecture comparisons.** Attention encoder-decoders [27], [28], [29] and Transformers [30] have been compared on character transduction with mixed outcomes [1], [4], [8], [9]. Moran and Lignos show that parameter count and weak statistics can reverse conclusions [9]; we therefore match size, tuning budget and seeds. Our work differs from all of the above in studying Tamil word transliteration under controlled, paired, severity-graded noise, with training remedies and architectures compared on the same footing.

## III. Data and Benchmark

**Training and clean evaluation.** We use the Dakshina Tamil romanization lexicon: 25,000 training words (68,218 romanization pairs), 2,500 development words (6,827 pairs) and 2,500 test words (6,864 pairs), which are word-disjoint [1]. Inputs are lowercase Latin and outputs are NFC-normalised Tamil. A romanization attested for several native forms is scored against all of them. The development set is used only for early stopping and tuning. The test lexicon holds 6,833 distinct romanized inputs.

**Independent test sets.** Aksharantar's Tamil test file contains every Dakshina test pair [5]. We keep only rows whose native word occurs in no Dakshina split, which gives three subsets by source: AK-Freq (2,607 distinct inputs, frequent web words), AK-NEI (991) and AK-NEF (706), plus their union AK-all. We also found that Aksharantar's training file shares stems with Dakshina test words, so we never train on it in the controlled experiments.

**Paired noisy test sets.** Seven operators perturb the romanized input only (Table I). Each of the 6,833 test inputs is perturbed at severity $p \in \{0.1, 0.2, 0.3, 0.5\}$ with a random seed derived from the word, operator and severity. The 28 sets are therefore exactly reproducible, and each noisy item stays paired with its clean form and references. Operators and rates are motivated by the typing phenomena in the literature [2], [3], [20]. The digraph operator swaps in both directions (th↔t, dh↔d, sh↔s, ch↔c,s) and maps zh to l or z only one way, because in our own measurement on public code-mixed Tamil comments [17] (heuristic, unverified by hand) vowel dropping, vowel-length collapse and stretching occur several times more often than in Dakshina, while dropped h does not. Table I reports realised strength: nominal $p$ is a per-position probability, so we report the share of words changed; the character edit rate at $p$=0.3 is 12.7% for V-DROP, 7.4% for KEY and 15.8% for MIX (see the repository manifest for all sets).

**Synthetic training data.** For the augmentation study we construct a pool of 60,000 further Tamil words from the IndicXlit unigram list. We remove every Dakshina word, every independent-test word and any word sharing a five-code-point prefix with a held-out word (46,652 of 137,748 candidates were removed for stem overlap alone). A Tamil-to-Latin BiLSTM trained on Dakshina romanizes each pool word

TABLE I
NOISE OPERATORS AND THE SHARE OF TEST WORDS THEY CHANGE (%) AT THREE SEVERITIES.

| Operator | Effect | Words changed (%) at $p$ = | | |
|---|---|---|---|---|
| | | 0.1 | 0.3 | 0.5 |
| V-DROP | delete non-initial vowels | 34 | 73 | 89 |
| V-LEN | collapse doubled vowels | 4 | 13 | 21 |
| GEM | collapse doubled consonants | 3 | 9 | 15 |
| DIGRAPH | th/t, dh/d, sh/s, ch/c/s swaps; zh→l,z | 12 | 32 | 47 |
| KEY | adjacent QWERTY key substitution | 21 | 52 | 69 |
| REPEAT | stretch one vowel by 1–3 letters | 10 | 29 | 51 |
| MIX | random combination of the six | 35 | 69 | 82 |

three times by sampling at temperature 1; a closeness filter against the model's greedy output drops 17,828 degenerate samples and leaves 123,488 pairs. A control set romanizes the same 60,000 words once, greedily (60,000 pairs). These romanizations are machine-made, so they carry the errors of the model that made them.

## IV. METHOD

**Models.** Both models are character-level, with separate source and target vocabularies built from the training data (30 and 52 symbols including specials). The BiLSTM has a two-layer bidirectional encoder, a two-layer LSTM decoder and Luong "general" attention [28], [29]. The Transformer is a pre-LN encoder-decoder with three layers per side, four heads and learned positions [30]. Width is chosen to hit about 1M, 3M and 8M trainable parameters: BiLSTM 0.98M, 2.99M and 8.04M; Transformer 0.80M, 3.15M and 8.70M, so "matched" means approximately matched.

**Training, tuning and decoding.** Adam with gradient clipping at 1.0 and early stopping on development CER (patience 6 during tuning). For each architecture and size we run 20 random-search trials over learning rate ($10^{-3.5}$ to $10^{-2.5}$), batch size (64–512), dropout, label smoothing (0–0.1) and, for the Transformer, warmup, selecting on development CER with beam 10 only; the test set is never used for selection. Decoding uses beam search with beam 10, which yields an $n$-best list.

**Metrics.** CER is the corpus-level Levenshtein distance over code points between the top hypothesis and the closest attested reference, divided by the reference length [1], [4]. Top-$k$ accuracy counts an input correct if any of the best $k$ hypotheses equals any reference, and mean reciprocal rank (MRR) uses the rank of the first correct one [13].

**Augmentation conditions.** A0 uses Dakshina only. A1 adds random character noise (deletion, insertion, substitution, swap) [21]. A2 adds the four Tamil-phonological operators V-DROP, V-LEN, GEM and DIGRAPH. A3 adds the 123,488 sampled back-transliterations [25]. A4 combines A2 and A3. CTRL adds the same pool words romanized once, greedily, and so separates "more words" from "more spelling variety". Online noise is re-drawn every epoch and leaves 40% of items clean. Two held-out variants, A2-noGEM and A2-noVDROP, omit one operator in training and are tested on it. Keyboard noise (KEY) and stretching (REPEAT) are deliberately absent from A2, so they test transfer.

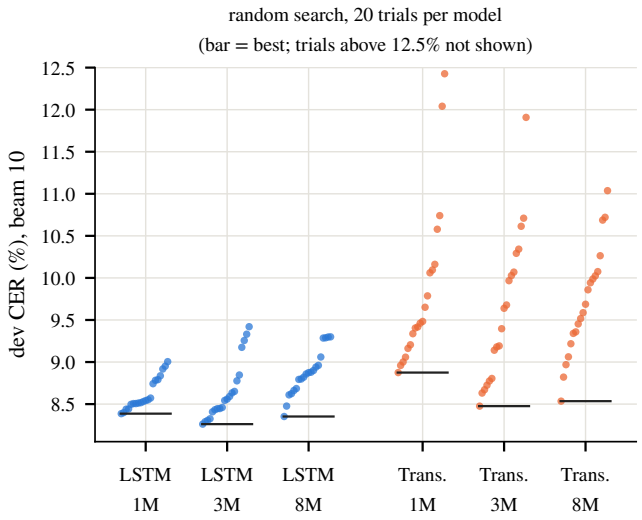



Fig. 1. Random-search tuning: development CER of 20 trials for each architecture and size. Bars mark the selected trial. Transformers are more sensitive to hyperparameters than BiLSTMs.

**Statistics.** Each cell is the mean and sample standard deviation over five seeds. System pairs are compared with a paired bootstrap over test items (10,000 resamples, seed-averaged item errors) and the six C2 conditions with Kruskal–Wallis and Dunn's test with Holm correction on per-seed CER. With five seeds the seed-level tests have little power, and the item bootstrap describes the test set rather than the population of trainings; we state this where it matters.

## V. EXPERIMENTAL SETUP

All models are implemented in PyTorch 2.11 (CUDA 12.8) and trained on a single RTX 5070 (12 GB) shared by up to three jobs. A final run trains for 7 to 32 minutes depending on the model, and the whole study (120 tuning trials, 30 comparison runs and 80 augmentation runs) took about one day of wall time. Every run writes a JSON file with its configuration, seed, code commit, data checksums and all metrics, and every number in this paper is generated from those files by scripts in the repository. The 110 final checkpoints were re-scored per item for the paired statistics; a stored model that did not reproduce its logged score would be flagged, and exactly one of 110 did (Section IX).

**Reproduction check.** Before any new experiment we verified our implementation against published numbers. The five-seed mean development CER of our tuned BiLSTM is 8.39–8.43, matching Roark et al.'s 8.4 [1], and the tuned 3M Transformer reaches 8.49, between their 9.4 and Kirov et al.'s 8.2 [4]. The untuned reproduction-gate runs (one seed, greedy) gave 8.95 and 9.38. Our top-1 accuracy (63.3%) is below the published 65.3% for the BiLSTM; we attribute this to differences in how multiple references and attestation counts are scored and do not hide it.

TABLE II

CLEAN RESULTS (MEAN ± SD OVER FIVE SEEDS). DEV AND TEST CER ARE IN %, WITH BEAM 10. BOLD: LOWEST TEST CER.

| Model | Size | Params | Dev CER | Test CER | Top-1 (%) | Top-5 (%) |
|---|---|---|---|---|---|---|
| BiLSTM | 1M | 0.98M | 8.39 ±0.11 | 8.90 ±0.24 | 63.6 ±0.5 | 82.5 ±0.4 |
| BiLSTM | 3M | 2.99M | 8.41 ±0.12 | 9.00 ±0.20 | 63.3 ±0.2 | 82.3 ±0.4 |
| BiLSTM | 8M | 8.04M | 8.43 ±0.06 | **8.77 ±0.07** | 63.4 ±0.2 | 83.0 ±0.4 |
| Transformer | 1M | 0.80M | 9.06 ±0.13 | 9.71 ±0.12 | 60.2 ±0.9 | 82.8 ±0.2 |
| Transformer | 3M | 3.15M | 8.49 ±0.03 | 9.01 ±0.17 | 63.1 ±0.7 | 83.4 ±0.4 |
| Transformer | 8M | 8.70M | 8.76 ±0.32 | 9.40 ±0.40 | 61.4 ±2.0 | 83.3 ±0.4 |

TABLE III

PAIRED BOOTSTRAP OVER 6,833 TEST ITEMS (10,000 RESAMPLES). ΔCER = CER($a$) − CER($b$); NEGATIVE FAVOURS $a$. INTERVALS THAT EXCLUDE ZERO ARE REAL FOR THIS TEST SET, NOT NECESSARILY FOR THE POPULATION OF TRAINED MODELS.

| System $a$ vs. $b$ | Suite | ΔCER ($a - b$) | 95% CI |
|---|---|---|---|
| BiLSTM 3M vs. Trans. 3M | clean | -0.01 | [-0.15, +0.12] |
| BiLSTM 3M vs. Trans. 3M | MIX $p$=0.3 | +1.30 | [+1.08, +1.53] |
| BiLSTM 8M vs. Trans. 8M | MIX $p$=0.3 | +0.28 | [+0.08, +0.48] |
| BiLSTM A2 vs. BiLSTM A1 | MIX $p$=0.3 | -0.70 | [-1.00, -0.41] |
| Trans. A2 vs. Trans. A1 | MIX $p$=0.3 | -0.03 | [-0.32, +0.27] |
| BiLSTM A3 vs. BiLSTM CTRL | clean | +0.03 | [-0.08, +0.13] |
| BiLSTM A3 vs. BiLSTM CTRL | MIX $p$=0.3 | -0.58 | [-0.75, -0.42] |
| Trans. A3 vs. Trans. CTRL | MIX $p$=0.3 | -0.81 | [-0.95, -0.66] |
| BiLSTM A4 vs. BiLSTM A2 | clean | -0.48 | [-0.62, -0.35] |
| Trans. A4 vs. Trans. A2 | clean | -0.83 | [-0.98, -0.68] |

## VI. RESULTS

All CER values are percentages on the test lexicon unless stated, with mean and standard deviation over five seeds.

### A. Clean accuracy and model size

Table II and Figure 3 (left) show clean results. At 3M parameters the BiLSTM and the Transformer are tied (test CER 9.00 and 9.01; paired-bootstrap difference −0.01, 95% CI [−0.15, +0.12]). At 1M and 8M the BiLSTM is better (by 0.81 and 0.63 CER; both intervals exclude zero), partly because the Transformer is less stable: its 8M model is worse than its 3M model (9.40 against 9.01) with the largest seed spread (sd 0.40, top-1 sd 2.0). The BiLSTM barely changes between 1M and 8M parameters (8.77 to 9.00), so on this word-level task capacity is not the limit. Top-5 accuracy is 82–83% for all models: the right answer is usually in the beam even when it is not first.

### B. RQ1: how much does typing noise hurt?

Figure 2 shows test CER against severity for the 3M models. Two operators cause most of the damage. Vowel dropping raises CER from about 9 to 24 at $p$=0.3 (+15.0 for the 1M BiLSTM, +12.6 for the 1M Transformer), and keyboard slips to about 22 (+12.4 to +13.1). Mixed noise is worst (24–25). Vowel-length collapse, doubled-consonant collapse and letter stretching cost only 1 to 2 points (+1.0 to +1.1 for V-LEN and GEM, +1.7 to +2.0 for REPEAT), and digraph swaps cost about 3.3. The damage grows steadily with severity: for MIX the 3M BiLSTM gives 14.8, 20.4, 25.5 and 34.4 at $p$ = 0.1, 0.2, 0.3 and 0.5. Keyboard slips were expected to be mild, but they are one of the two most harmful operators; per edited character they hurt more than vowel dropping (KEY edits 7.4% of characters at $p$=0.3 for +12.9 CER, V-DROP 12.7% for +15.0 CER for the 1M BiLSTM).

### C. RQ4: which architecture degrades less?

Under mixed noise at $p$=0.3 the Transformer is better at every size (BiLSTM minus Transformer CER: +0.42 at 1M, +1.30 at 3M, +0.28 at 8M; Table III), and also degrades less in relative terms (+168% against +183% at 3M), most clearly under vowel dropping (21.7 against 24.0 at 3M; Figure 3). On clean text the BiLSTM is equal or better. There is thus a small trade-off, but the gaps are 0.3 to 1.3 CER, of the order of the effects of tuning and seed, so we do not claim that the Transformer is generally more robust. The item-level bootstrap resamples test items and averages seeds, so its very small $p$ values describe the test set, not the population of trainings.

### D. RQ3: which training augmentation helps?

Table IV and Figure 4 compare the six conditions for both 3M architectures. **Noise training buys robustness and costs clean accuracy.** Random noise (A1) and Tamil-phonological noise (A2) lower mixed-noise CER by 2.3 and 3.0 for the BiLSTM (1.8 each for the Transformer) but add 0.8 to 0.9 clean CER, more than a 0.5 budget we set in advance. Part of this cost may come from selecting on clean development CER, which favours the clean model. **Tamil-aware noise helps only for what it models.** A2 beats A1 on vowel dropping (17.7 against 21.7) but is worse on keyboard slips (23.4 against 18.6), which A2 does not model. For the Transformer A2 and A1 are indistinguishable on mixed noise (−0.03, CI [−0.32, +0.27]).

**Extra words, not spelling variety, explain the clean gain of back-transliteration.** A3 improves clean CER over A0 (8.67 against 9.00 for the BiLSTM), but so does the control with one fixed romanization per word (8.64), and A3 and CTRL are indistinguishable on clean text (+0.03, CI [−0.08, +0.13]). The sampled variants add a small robustness gain on mixed noise (−0.58 for the BiLSTM, −0.75 for the Transformer; Table III). **The combination is best.** A4 has the lowest mixed-noise CER for both architectures (21.8) and removes most of A2's clean cost (+0.35 for the BiLSTM, −0.03 for the Transformer). Our design has no "A2 plus control" condition, so we cannot say how much of A4's noise gain is due to varied romanizations and how much to the extra words.

**Robustness does not transfer to a held-out operator.** Leaving vowel dropping out of A2 removes almost all of its benefit: BiLSTM CER on V-DROP is 24.97 against 17.70 for full A2, which is even slightly worse than the clean baseline A0 (24.03); the Transformer shows the same pattern (22.35 against 17.43 and 21.65; Figure 5). Leaving out GEM costs only 0.15 to 0.23 CER on GEM, but GEM is mild to begin with. A model trained on one error type does not become robust to another, in line with earlier findings [20].

**Independent sets.** On AK-all, A1 and A2 are worse than A0 (13.7 and 12.7 against 11.7 for the BiLSTM), while A3,

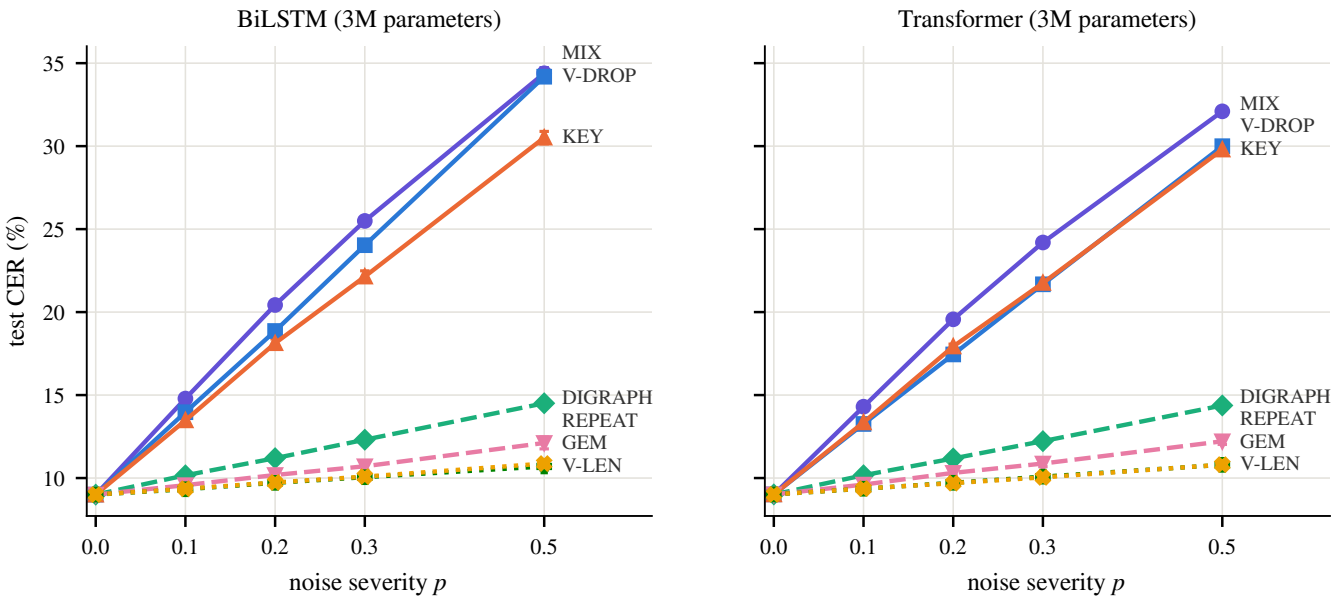



Fig. 2. Test CER of the 3M models against noise severity for the seven operators. Vowel dropping, keyboard slips and mixed noise damage the models far more than vowel-length, doubled-consonant and stretching noise. Error bars: sd over five seeds.

TABLE IV

AUGMENTATION CONDITIONS AT 3M PARAMETERS: TEST CER (%) ON CLEAN TEXT, MIXED NOISE ($p$=0.3), VOWEL DROPPING ($p$=0.3), KEYBOARD SLIPS ($p$=0.3) AND THE INDEPENDENT AK-ALL SET (MEAN OVER FIVE SEEDS; SD AT MOST 0.48; BOLD: LOWEST PER COLUMN AND ARCHITECTURE).

| | BiLSTM 3M | | | | | Transformer 3M | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Condition | clean | MIX | V-DR. | KEY | AK | clean | MIX | V-DR. | KEY | AK |
| A0 | 9.00 | 25.49 | 24.03 | 22.14 | 11.67 | 9.16 | 24.27 | 21.65 | 21.91 | 11.91 |
| A1 | 9.90 | 23.17 | 21.69 | **18.58** | 13.73 | 9.94 | 22.49 | 20.45 | **18.23** | 13.53 |
| A2 | 9.83 | 22.47 | 17.70 | 23.43 | 12.74 | 9.96 | 22.46 | 17.43 | 23.73 | 12.95 |
| A3 | 8.67 | 25.10 | 22.91 | 21.83 | 10.78 | **8.65** | 23.83 | 21.11 | 21.08 | **10.38** |
| A4 | 9.35 | **21.82** | **16.65** | 22.85 | 11.32 | 9.13 | **21.79** | **16.39** | 22.69 | 11.02 |
| CTRL | **8.64** | 25.68 | 23.58 | 22.09 | **10.67** | 8.67 | 24.63 | 21.70 | 21.91 | 10.40 |

CTRL and A4 are better or equal (10.8, 10.7 and 11.3). The A3 and CTRL words come from the same kind of word list as Aksharantar, so part of that gain may be vocabulary overlap rather than robustness. Kruskal–Wallis tests over the six conditions are significant for both architectures on clean and mixed noise (effect size $\eta^2$ = 0.88 to 0.93), and Dunn's test with Holm correction separates the noise-only conditions (A1, A2) from the data-rich ones (A3, CTRL) on clean text, and A4 from A0 and CTRL on mixed noise; neighbouring conditions such as A2 and A1 for the Transformer are not separated with five seeds.

## VII. DEPLOYMENT MODEL

The experiments above compare many small models; for use we also want a single model. We take the best seed of the A4 Transformer (3.15M parameters) and fine-tune *all* of its weights, with no adapters and no frozen layers, so the result is the same architecture in one 12.6 MB file. The training mixture has 2.41M pairs per epoch: Dakshina train repeated three times, the 123,488 back-transliterated pairs and 2.08M Aksharantar pairs, from which we dropped IndicCorp-mined rows below the authors' quality score of −0.10 (1.15M of 3.23M rows) and every word of the Dakshina and independent test sets. Noise from all six operators is applied online to half of the items. We use Adam with learning rate $10^{-4}$ (cosine decay, 500 warmup steps), batch size 512, label smoothing 0.1, five epochs (20 minutes on one GPU) and keep the epoch with the best mean of clean and mixed-noise development CER.

Table V compares the result with the A4 model it started from. Clean accuracy is unchanged within noise (test CER 8.93 against 9.02, top-1 +1.4 points, top-5 −0.2) and the independent Aksharantar sets improve slightly (AK-all 10.76 against 10.95, every subset lower). Robustness improves strongly: mixed-noise CER at $p$=0.3 falls from 21.80 to 17.77 and keyboard-slip CER from 23.04 to 15.40. These noise gains are *not* held-out evidence: the fine-tuning mixture contains all six operators, including KEY and REPEAT that A2 lacked, and the noisy test sets are built from the same operators, so the gain measures what the model learned to ignore and not generalisation to unseen error types (Section VI-D). The

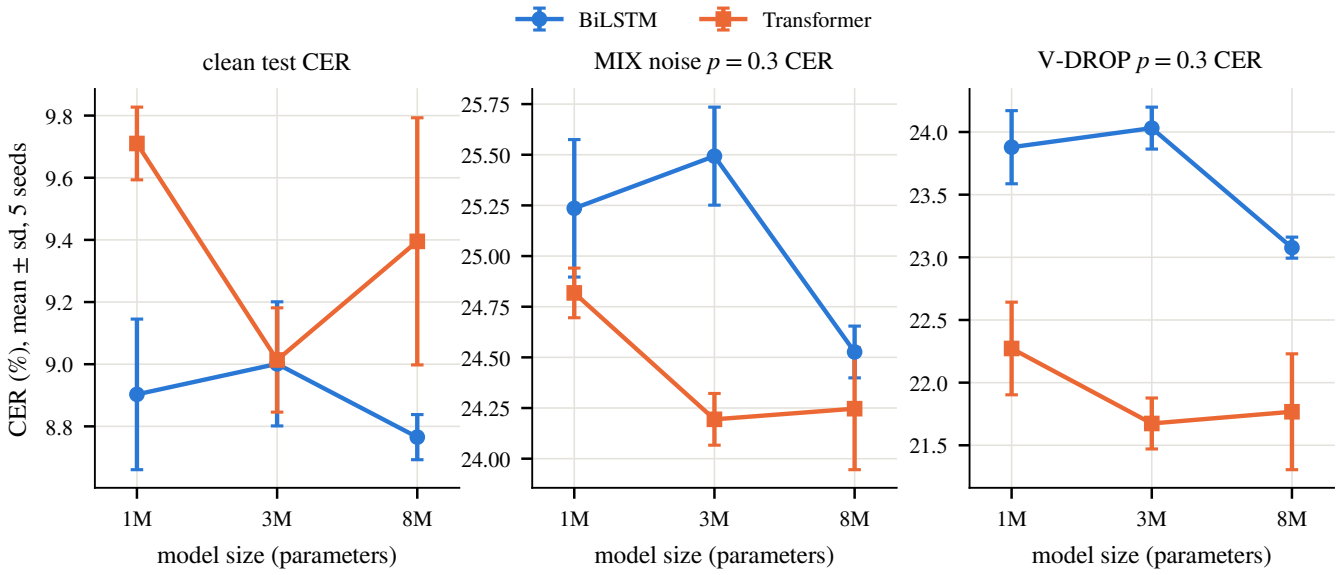



Fig. 3. Clean, mixed-noise and vowel-drop test CER against model size (mean ± sd over five seeds). The architectures tie on clean text at 3M; under noise the Transformer is lower at every size.

TABLE V
DEPLOYMENT MODEL AGAINST THE A4 TRANSFORMER IT WAS FINE-TUNED FROM (TEST CER IN %, BEAM 10, SINGLE RUN EACH).

| Test suite | v1 (A4) | v2 (fine-tuned) | Δ |
|---|---|---|---|
| Dev CER | 8.47 | 8.56 | +0.10 |
| Test CER | 9.02 | 8.93 | -0.09 |
| Test top-1 (%) | 63.88 | 65.27 | +1.39 |
| Test top-5 (%) | 85.03 | 84.81 | -0.22 |
| AK-all CER | 10.95 | 10.76 | -0.20 |
| MIX $p$=0.1 CER | 13.41 | 11.89 | -1.52 |
| MIX $p$=0.3 CER | 21.80 | 17.77 | -4.03 |
| MIX $p$=0.5 CER | 28.79 | 23.02 | -5.77 |
| V-DROP $p$=0.3 CER | 16.20 | 16.00 | -0.20 |
| KEY $p$=0.3 CER | 23.04 | 15.40 | -7.63 |
| REPEAT $p$=0.3 CER | 10.97 | 9.51 | -1.46 |

2.1M mined pairs give a much smaller clean-accuracy gain than adding data did in the controlled experiments, which suggests that the mined romanizations resemble the existing data (formal text, careful spelling) and that a 3M-parameter model has little room left on them.

**Preliminary real-chat check.** We also scored both models on 370 words taken from the author's private chat exports, labelled with Tamil spellings by a language model; *these labels are not yet verified by a person*, so the numbers are only indicative. CER is 25.4 for the A4 model and 24.9 for the fine-tuned model (top-1 40.5% and 39.7%, top-5 68.6% and 71.1%): the two are indistinguishable, and both are far above the clean 9 and close to our heavy synthetic noise ($p$=0.3 mixed noise: 18–24). Colloquial spellings such as *poren* (போறேன்) are the typical failures, because mined and Wikipedia-style data use literary forms. This gap is the main open problem and the reason we regard real-typing evaluation as the next step.

## VIII. Discussion

**What the noise tells us about Tamil.** Vowel omission and key slips destroy the information the model needs. Dropped vowels make the input genuinely ambiguous (a string like *kankl* fits several Tamil words), so part of the remaining error is irreducible without context or a lexicon; top-5 accuracy and language-model reranking are therefore natural next steps rather than larger models. By contrast, vowel-length and doubled-letter errors cost only 1 to 2 CER points, possibly because the Dakshina training data already contain many romanizations that collapse these distinctions [1], [2]; we did not test this. It suggests that a Tamil-aware noise model should concentrate on vowel omission and keyboard errors.

**Data, not capacity.** Increasing the BiLSTM from 1M to 8M parameters changes clean test CER by at most 0.25, while adding 60,000 pool words (A3 and CTRL) lowers it by 0.3 to 0.5 and the independent-set CER by about 1 to 1.5. On this task the limiting factor is the amount and variety of romanized data. This is consistent with the large gain that Aksharantar reports from mined data [5], and it motivates the larger-data fine-tuning described in Section VII.

**Back-transliteration: words or variants?** Previous work on romanized language identification found that sampled spelling variants beat larger models [22]. Our control isolates the factors for transliteration: the clean-accuracy gain comes from extra *words*, while spelling variants add a smaller robustness gain on mixed noise. Because the variants come from a model trained on Dakshina, they reproduce Dakshina-style variation and not chat-style variation, so a larger effect may need variants that reflect real typing.

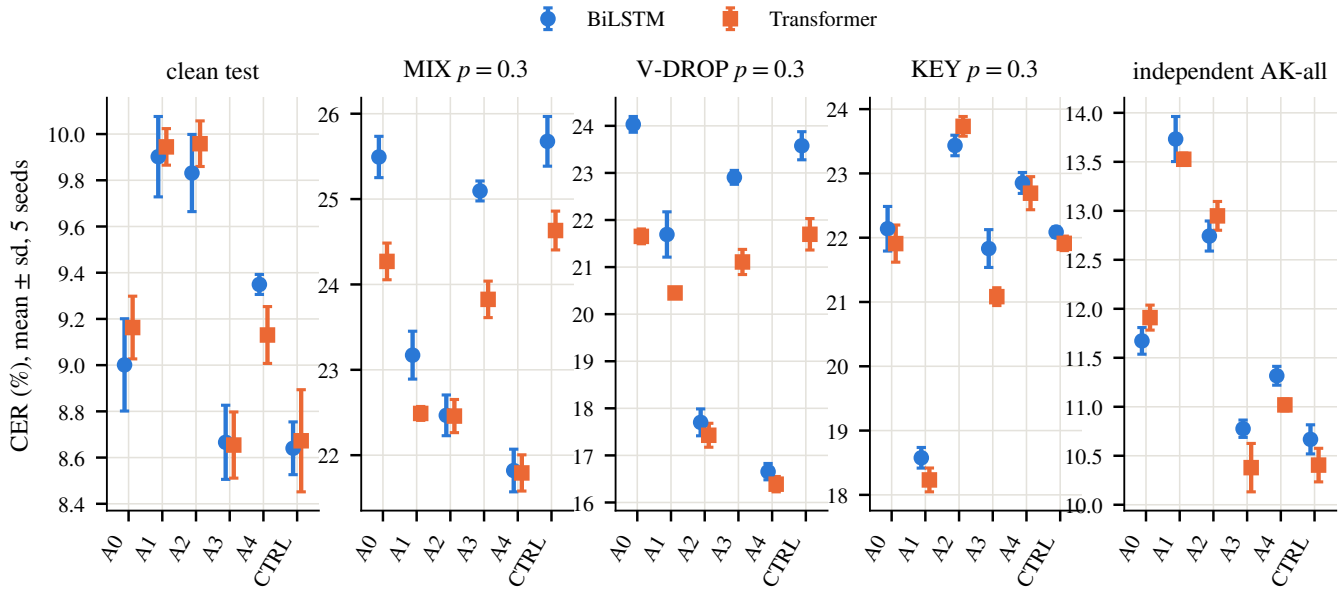



Fig. 4. Augmentation conditions: test CER on clean text, three noise suites and the independent AK-all set (mean ± sd over five seeds).

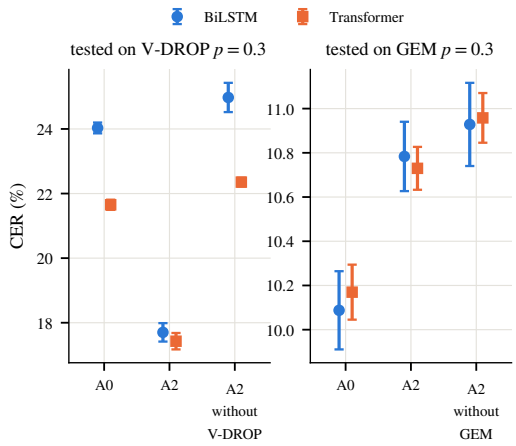



Fig. 5. Held-out operators. Removing V-DROP from A2's training removes its benefit on V-DROP; removing GEM changes little.

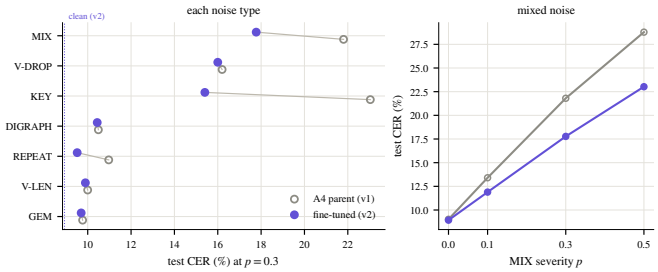



Fig. 6. Fine-tuned model (v2) against its A4 parent (v1) on each noise type at $p$=0.3 (left) and on mixed noise at all severities (right). The largest gains are on keyboard slips and mixed noise; all six operators were in the fine-tuning mixture, so this is not held-out evidence.

**Operator-specific robustness.** Training on V-DROP gives a large gain on V-DROP and none on a held-out operator, as Belinkov and Bisk found for natural language noise [20]. Practical systems therefore need noise operators that cover the phenomena users produce, and the benchmark we release is a way to check coverage rather than a replacement for real data.

**Architectures.** The conflict between Roark et al. and Kirov et al. is largely a matter of tuning and seeds: with 20 tuned trials per cell and five seeds the BiLSTM and the Transformer tie at 3M parameters, and the Transformer leads only under heavy noise. Our tuning showed that Transformers are the more sensitive of the two to hyperparameters (Figure 1), which supports the view that untuned comparisons can go either way [8], [9].

## IX. Limitations and Threats to Validity

**No real typing yet.** All noise is synthetic and applied to Dakshina romanizations, which are formal text typed with care, not chat. We built a real-chat word set from private message exports and a protocol for a team-typed set. Only a preliminary check of the real-chat set exists (Section VII) and its labels are not yet verified by a person, and the team-typed set has not been collected, so the question of whether synthetic noise predicts real typing (our RQ2) is open. We therefore speak of informal *romanized* Tamil and make no claim about chat text.

**Synthetic noise and training data.** The operators and their rates come from the literature and from measurements on public code-mixed corpora, not from a hand-checked sample of real typing. Back-transliterated pairs are machine-made and inherit the errors of the generating model. Operators in the

noisy test sets are the same ones that A2 trains on, except KEY and REPEAT, so only these two measure transfer.

**Tuning and statistics.** Hyperparameters were tuned once per architecture and size on clean development CER with one seed, and noise-trained conditions reuse the tuned clean settings; early stopping uses clean development CER, which disadvantages noise-trained models. Five seeds give low power for small differences, and the paired bootstrap does not model seed variation. Parameter counts are matched only approximately.

**An incident.** A bug in our job supervisor started duplicate worker processes overnight, and 33 of the first 72 finished augmentation runs were trained while a duplicate process trained the same run and shared its checkpoint folder. We checked the damage three ways. A clean rerun of one affected BiLSTM run reproduced the stored result exactly; a clean rerun of one affected Transformer run differed slightly (test CER 9.30 against 9.13), which is within normal seed variation for a non-deterministic model; and a per-item re-scoring of all 110 stored checkpoints found one model that did not reproduce its logged score (8.43 against 8.49 CER). That run was retrained cleanly and all other results were kept; the scripts, logs and the incident report are in the repository.

**Privacy.** The real-chat words come from the author's own private conversations with other people. Only a list of distinct lowercase words with counts was extracted (no messages, names or timestamps), the list stays on the author's machines and was labelled without sender information, and only aggregate scores are reported here. The data cannot be released.

**Other.** Dakshina's Tamil lexicon contains about 0.35% bad pairs, which we did not remove from the test set. Vowel-dropped inputs can be genuinely ambiguous, so top-1 accuracy understates what a user-facing system with alternatives achieves. Our models are word-level; no sentence context is used.

## X. Conclusion

We built a paired, seeded robustness benchmark for Latin-to-Tamil transliteration and used 110 trained models to answer three questions. Typing noise hurts mainly through dropped vowels and key slips; Tamil-aware noise training removes most of the vowel-drop damage but not damage from operators it does not model; extra romanized words matter at least as much as spelling variety, and the combination of Tamil-phonological noise with back-transliterated data is the best overall at no clean cost; and the BiLSTM and the Transformer are close, with the Transformer slightly better under heavy noise. Our implementation reproduces the published Dakshina baselines.

**Future work.** The next step is to evaluate on real typing: a verified real-chat word set and a team-typed set, which will show whether synthetic noise predicts real degradation. Other directions are lexicon and language-model reranking on noisy input, sampling spelling variants from real typing rather than from a Dakshina-trained model, byte-level pretrained models, and sentence-level context.

**Reproducibility.** Code, configurations, per-run logs, result summaries and the scripts that generate every table and figure are in the project repository (`github.com/lebiraja/tamil-translit-robustness`, private until release); the Dakshina and Aksharantar data are available from their original sources under their own licences, and the derived noisy test sets are regenerated deterministically by the repository scripts.

## References


[1] B. Roark et al., "Processing South Asian languages written in the Latin script: The Dakshina dataset," in *Proceedings of the 12th Conference on Language Resources and Evaluation (LREC 2020)*, Marseille, France: European Language Resources Association, 2020, pp. 2413–2423.

[2] L. Wolf-Sonkin et al., "Latin script keyboards for South Asian languages with finite-state normalization," in *Proceedings of the 14th International Conference on Finite-State Methods and Natural Language Processing*, Dresden, Germany, 2019, pp. 108–117.

[3] H. M. A. D. Herath and T. G. D. K. Sumanathilaka, "TAMZHI: Shorthand romanized Tamil to Tamil reverse transliteration using novel hybrid approach," *International Journal on Advances in ICT for Emerging Regions (ICTer)*, vol. 17, no. 1, pp. 1–7, 2024. DOI: 10.4038/icter.v17i1.7271

[4] C. Kirov et al., "Context-aware transliteration of romanized South Asian languages," *Computational Linguistics*, vol. 50, no. 2, pp. 475–534, 2024. DOI: 10.1162/coli_a_00510

[5] Y. Madhani et al., "Aksharantar: Open Indic-language transliteration datasets and models for the next billion users," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, Singapore, 2023, pp. 40–57.

[6] B. Baiju et al., *Romanized to native Malayalam script transliteration using an encoder-decoder framework*, IndoNLP 2025 shared-task system (team NexText), 2024. arXiv: 2412.09957 [cs.CL].

[7] G. Azam et al., *Beyond specialization: Benchmarking LLMs for transliteration of Indian languages*, 2025. arXiv: 2505.19851 [cs.CL].

[8] S. Wu, R. Cotterell, and M. Hulden, *Applying the transformer to character-level transduction*, 2021. arXiv: 2005.10213 [cs.CL].

[9] M. Moran and C. Lignos, "Effective architectures for low resource multilingual named entity transliteration," in *Proceedings of the 3rd Workshop on Technologies for MT of Low Resource Languages*, 2020, pp. 79–86.

[10] A. Kunchukuttan, R. Puduppully, and P. Bhattacharyya, "Brahmi-Net: A transliteration and script conversion system for languages of the Indian subcontinent," in *Proceedings of NAACL-HLT 2015: Demonstrations*, Denver, Colorado, 2015, pp. 81–85.

[11] A. Kunchukuttan et al., "Leveraging orthographic similarity for multilingual neural transliteration," *Transactions of the Association for Computational Linguistics*, vol. 6, pp. 303–316, 2018.

[12] A. Kunchukuttan, S. Jain, and R. Kejriwal, "A large-scale evaluation of neural machine transliteration for Indic languages," in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, 2021, pp. 3469–3475.

[13] N. Chen et al., "Report of NEWS 2018 named entity transliteration shared task," in *Proceedings of the Seventh Named Entities Workshop*, Melbourne, Australia, 2018, pp. 55–73.

[14] L. Xue et al., "ByT5: Towards a token-free future with pre-trained byte-to-byte models," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 291–306, 2022.

[15] L. Hellsten et al., "Transliterated mobile keyboard input via weighted finite-state transducers," in *Proceedings of the 13th International Conference on Finite State Methods and Natural Language Processing (FSMNLP 2017)*, Umeå, Sweden, 2017, pp. 10–19. DOI: 10.18653/v1/W17-4002

[16] D. Sumanathilaka et al., *IndoNLP 2025: Shared task on real-time reverse transliteration for romanized Indo-Aryan languages*, 2025. arXiv: 2501.05816 [cs.CL].

[17] B. R. Chakravarthi et al., *Corpus creation for sentiment analysis in code-mixed Tamil-English text*, 2020. arXiv: 2006.00206 [cs.CL].

[18] S. Mandal and K. Nanmaran, *Normalization of transliterated words in code-mixed data using Seq2Seq model & Levenshtein distance*, 2018. arXiv: 1805.08701 [cs.CL].

[19] C. Gulcehre et al., *On using monolingual corpora in neural machine translation*, 2015. arXiv: 1503.03535 [cs.CL].

[20] Y. Belinkov and Y. Bisk, "Synthetic and natural noise both break neural machine translation," in *International Conference on Learning Representations (ICLR)*, 2018.

[21] V. Karpukhin et al., *Training on synthetic noise improves robustness to natural noise in machine translation*, 2019. arXiv: 1902.01509 [cs.CL].

[22] A. Benton et al., *Improving informally romanized language identification*, 2025. arXiv: 2504.21540 [cs.CL].

[23] Y. Madhani, M. M. Khapra, and A. Kunchukuttan, "Bhasha-Abhijnaanam: Native-script and romanized language identification for 22 Indic languages," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, Toronto, Canada, 2023, pp. 816–826.

[24] K. Gharami et al., *Modeling romanized Hindi and Bengali: Dataset creation and multilingual LLM integration*, 2025. arXiv: 2511.22769 [cs.CL].

[25] R. Sennrich, B. Haddow, and A. Birch, *Improving neural machine translation models with monolingual data*, 2016. arXiv: 1511.06709 [cs.CL].

[26] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Berlin, Germany, 2016, pp. 1715–1725.

[27] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Advances in Neural Information Processing Systems*, 2014, pp. 3104–3112.

[28] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *International Conference on Learning Representations (ICLR)*, 2015.

[29] M.-T. Luong, H. Pham, and C. D. Manning, "Effective approaches to attention-based neural machine translation," in *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2015.

[30] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.